

Chapter 4: Functions and Program Structure

Relational graph edition. Pull-shaped. Top-down.

How to use this document

Red = checkpoint. Green = practice. Blue = target. Walk edges in order.

Big picture

Functions decompose programs into named, reusable operations. Scope rules determine which names are visible where. The preprocessor runs before compilation and does text substitution. This chapter turns you from someone who writes one big `main()` into someone who designs programs.

Target problem

Build a reverse-Polish calculator with: a main loop, separate push/pop functions for a value stack, a getop function that reads tokens, a #define for stack size, and a recursive factorial function for testing.

The Graph

Node inventory

Node	What it is	Key rule
function definition	<code>return-type name(params) { body }</code>	The actual code that runs
function declaration/prototype	<code>return-type name(params);</code>	Tells compiler the signature exists (no body)
extern	<code>extern type name;</code>	Variable/function defined in another file
static (file scope)	<code>static type name;</code>	Limits visibility to this file only
static (local)	<code>static type name; inside function</code>	Value persists between calls
register	<code>register type name;</code>	Hint: keep in CPU register (compiler may ignore)
#include	<code>#include <file> or "file"</code>	Copies file contents into this position
#define	<code>#define NAME(args) body</code>	Text substitution macro, can take arguments
#ifdef / #ifndef	<code>#ifdef NAME ... #endif</code>	Conditional compilation

recursion	function calls itself	Needs base case to terminate
-----------	-----------------------	------------------------------

Edge inventory

Edge	Rule
E1: Scope	A name is visible from its declaration to the end of its enclosing block/file
E2: Linkage	extern = visible across files. static = visible only in this file
E3: Declaration before use	Functions must be declared (prototype) before they are called
E4: Preprocessor order	#include runs first, then #define expands, then #ifdef evaluates, then compiler sees result
E5: Call stack	Each function call pushes a frame (locals + return address). Return pops it
E6: Recursion	Function calls itself with smaller input until base case returns directly

Zoom in -- one edge at a time

Edge 1: Function basics

A function has four parts: return type, name, parameter list, body. If no return type is specified, old C assumed int (modern C requires explicit type). A function that returns nothing uses void.

```
/* Declaration (prototype) -- promises this function exists */
double celsius(int fahr);

/* Definition -- the actual implementation */
double celsius(int fahr) {
    return (5.0 / 9.0) * (fahr - 32);
}
```

Problem 4.1: Write a function `int abs_val(int x)` that returns the absolute value of `x`.

Answer: `int abs_val(int x) { return (x < 0) ? -x : x; }`

Edge 2: Scope -- where names live

Local variables: visible only inside their function. Die when function returns. External variables: defined outside all functions. Visible from definition to end of file. Block variables: declared inside `{ }`, visible only within that block.

Scope hierarchy: block $\subset$ function $\subset$ file $\subset$ program

Inner scopes can shadow outer names. Avoid this -- it creates confusion.

Problem 4.2: In the code below, what value does each `printf` output?

```
int x = 1; { int x = 2; printf("%d", x); } printf("%d", x);
```

Answer: 2 then 1. The inner `x` shadows the outer `x` inside the block.

Edge 3: static -- controlling visibility and lifetime

Two uses of static: (1) On an external variable or function: limits visibility to this file (other files cannot see it even with `extern`). (2) On a local variable: value persists between calls (not destroyed when function returns).

```
/* File-scope static: invisible to other files */
static int module_counter = 0;

/* Local static: persists between calls */
int next_id(void) {
    static int id = 0;
    return ++id; /* returns 1, 2, 3, ... on successive calls */
}
```

Problem 4.3: What does `next_id()` return on its 5th call?

Answer: 5. The static local id persists and increments each call.

CHECKPOINT A

Without looking: what is the difference between a declaration and a definition? What are the two meanings of static? What is extern for?

Edge 4: Header files and program organization

Put shared declarations (prototypes, #defines, type definitions) in .h files. Put implementations in .c files. Each .c file #includes the headers it needs. This prevents duplication and ensures all files agree on function signatures.

```
/* calc.h -- shared declarations */
#define MAXVAL 100
void push(double);
double pop(void);
int gettop(char []);
```

Problem 4.4: Why should you NOT put variable definitions (like `int x = 5;`) in a header file?

Answer: Every .c file that #includes the header gets its own copy of x, causing linker errors (multiple definitions of the same symbol).

Edge 5: The C preprocessor

Runs BEFORE the compiler. Three key operations: (1) #include copies file contents in. (2) #define does text replacement (can take arguments). (3) #ifdef/#ifndef enables conditional compilation.

```
#define MAX(a, b) ((a) > (b) ? (a) : (b))
#define SQUARE(x) ((x) * (x))

/* Parentheses are critical! Without them: */
/* SQUARE(1+2) becomes ((1+2) * (1+2)) = 9 (correct) */
/* Without parens: 1+2 * 1+2 = 5 (wrong!) */
```

Macro trap: Arguments are substituted textually, not evaluated. Always parenthesize every parameter and the whole expression.

Problem 4.5: What does `SQUARE(i++)` expand to? Why is this a problem?

Answer: `((i++) * (i++))` -- i is incremented twice! Side effects in macro arguments are dangerous.

Edge 6: Recursion

A function calls itself with a smaller input until reaching a base case. Each call pushes a new stack frame. The base case returns without recursing, and the stack unwinds.

```
long factorial(int n) {
    if (n <= 1) return 1; /* base case */
    return n * factorial(n - 1); /* recursive case */
}
```

Problem 4.6: Trace `factorial(4)`: how many stack frames are created? What value does each return?

Answer: 4 frames. `factorial(1)=1`, `factorial(2)=2`, `factorial(3)=6`, `factorial(4)=24`.

CHECKPOINT B

Write a recursive function `void printd(int n)` that prints an integer as a string of digits. (Hint: print the leading digits recursively, then the last digit.)

Solving the target problem

Walk: E4 (header with shared declarations) + E1 (push/pop/getop functions) + E2 (scope for stack) + E6 (factorial for testing)

```
/* Simplified RPN calculator */
#include <stdio.h>
#include <stdlib.h> /* for atof */
#include <ctype.h>
#define MAXOP 100
#define NUMBER 0
#define MAXVAL 100

static int sp = 0; /* stack pointer */
static double val[MAXVAL]; /* value stack */

void push(double f) {
    if (sp < MAXVAL) val[sp++] = f;
    else printf("error: stack full\n");
}

double pop(void) {
    if (sp > 0) return val[--sp];
    printf("error: stack empty\n");
    return 0.0;
}
```

Edges walked: E2 (static file-scope for `sp` and `val` -- hidden from other files), E1 (push/pop as separate functions), E5 (`#define` for constants).

CHECKPOINT C

Add a long `factorial(int n)` recursive function to the calculator. When the user types 'f', pop the top value, compute its factorial, and push the result. Report your code.

Final exam

Problem 1: Add modulus (%) support to the RPN calculator for integer operands.

Problem 2: Write a macro `#define SWAP(t, x, y)` that swaps two variables of type `t`.

Problem 3: Add an 'undo' command to the calculator that restores the stack to its state before the last operation.

Problem 4: Rewrite `printd` (recursive digit printer) to handle the most negative integer correctly.

Problem 5: Write a `#ifndef` include guard for `calc.h` and explain why it is needed.

CHECKPOINT D

Solve at least 3. Report in chat.

Answer key

1. Walk E1: pop two values, cast to int, compute `a%b`, push result. Handle `b==0`.
2. `#define SWAP(t,x,y) do { t _tmp = (x); (x) = (y); (y) = _tmp; } while(0)`. The `do-while(0)` makes it safe in if-else.
3. Walk E3: save `sp` to a static `prev_sp` before each operation. 'undo' restores `sp = prev_sp`.
4. Walk E6: `INT_MIN` cannot be negated (overflow). Handle it specially by printing the last digit of the positive remainder.
5. `#ifndef CALC_H / #define CALC_H / ... / #endif`. Without it, including `calc.h` twice causes duplicate declarations.